

Micro não aguenta 98 números. E já não aguenta os 3 de hoje.

Medição da produção em 31 de agosto de 2026, com o alvo de setembro: ~100 usuários simultâneos e ~98 números de WhatsApp conectados, distribuídos por várias organizações.

Projeto	Compute atual	Estado hoje	Alvo
zaobtetbjpuzibjymhzw	Micro · 1 GB · CPU compartilhada	3 números · 4 orgs · 7 usuários	98 números · 100 usuários

572 MB
de swap no pico, numa máquina de 906 MB. O mínimo registrado é 295 MB — **nunca zero**.

72%
de todo o tempo de banco do mês gasto pelo coletor de lixo do `pg_net`, não pelo seu código.

283 MB
ocupados pela tabela `net._http_response` para guardar 2.160 linhas. 26% do banco.

2–15%
de uso de disco I/O. O disco é a única coisa folgada — o gargalo não é ele.

Veredito

O Micro precisa ser trocado — mas a troca sozinha resolve o sintoma errado.

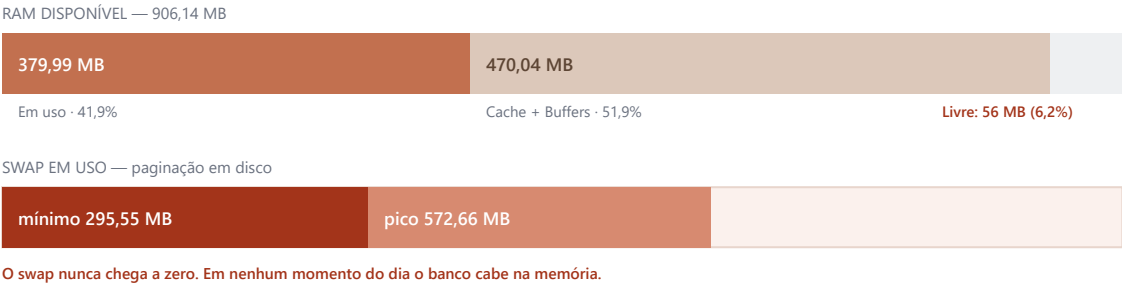
O banco tem 1,08 GB e a máquina tem 906 MB de RAM: o Postgres pagina em disco 24 horas por dia, com 3 números conectados e 1,8 mensagem por minuto. Ao mesmo tempo, **72% do trabalho do banco não é seu** — é o `pg_net` varrendo uma tabela de 283 MB que guarda 2.160 linhas úteis. Limpar essa tabela devolve 26% do banco de graça e pode acabar com o swap hoje. Comprar uma máquina maior antes disso é comprar capacidade para varrer lixo mais rápido.

Recomendação: limpar o `pg_net` e desligar os crons ociosos primeiro (custo zero, efeito imediato), depois subir para **compute Large** — 8 GB, CPU dedicada, **+US\$ 100/mês** — e só então montar o staging e rodar a carga. Detalhes e ordem de execução nas próximas páginas.

O retrato da produção em 31/08/2026

Coletados no painel do Supabase (Reports, Usage, Query Performance) e por consulta direta ao banco. Período de faturamento: 03/ago – 03/set. Picos, não médias.

Memória — pico de 25/ago às 13h



Conexões — pico de 36 em 60 (60%)

Categoria	Valor
Postgres	20
Res.	9
Outros papéis	2
livre	24

Storage 4 · Auth 1 · Outros papéis 2. O papel “postgres” (20) é dos crons e do pg_net.

60% do teto com 3 números. O Large sobe de 60 para 160 conexões diretas, e de 200 para 800 no pooler.

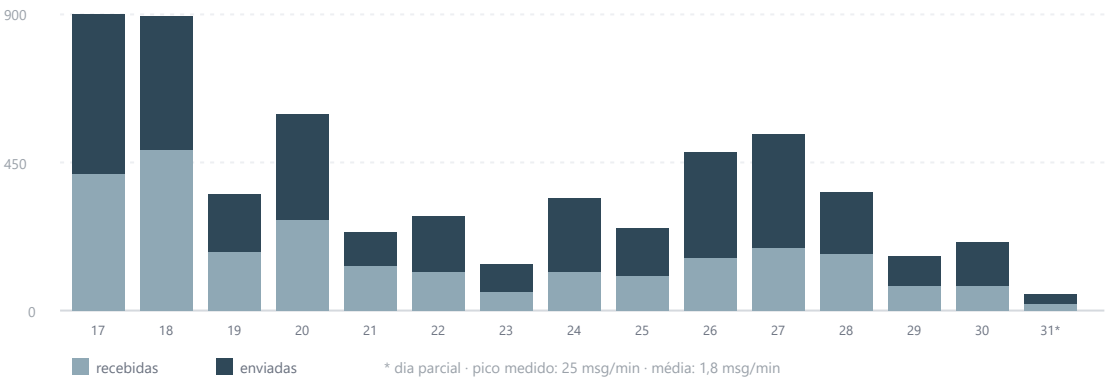
CPU e disco — os dois folgados

CPU — pico 4,15%, nunca acima de 50%

Disk I/O — variação de 2% a 15%

A máquina não está sem CPU nem sem disco. Está sem memória.

Volume real de mensagens — 17 a 31 de agosto



MÉTRICA	MEDIDO	LEITURA
Swap em uso	295–572 MB	Banco de 1,08 GB numa máquina de 906 MB. Nunca zero.
Cache hit ratio	0,9830	Abaixo de 0,99 confirma falta de RAM, de forma independente do swap.
Conexões (pico)	36 / 60	60% do teto com 3 números. Escala com crons e instâncias.
CPU (pico)	4,15%	Folgado. Nunca passa de 50% em nenhuma hora do dia.
Disk I/O	2–15%	Folgado. O crédito de burst não é o gargalo hoje.
Tamanho do banco	1,08 GB	De 8 GB de disco provisionado. 283 MB são lixo do pg_net.
Invocações de edge function	305.856	De 2 M inclusas. ~93% são crons — ver seção 3.
Realtime — pico simultâneo	5	De 500 inclusas. Com 100 usuários vira algumas centenas (6 canais por usuário na tela de chat).
Realtime — mensagens	45.209	De 5 M inclusas. Folgado.
Egress	5,29 GB	De 250 GB inclusos. Folgado.
Usuários ativos no mês	7 MAU	Alvo: 100 simultâneos.

72% do seu banco de dados não trabalha para você

Fonte: Advisors → Query Performance, aba “Most time consuming”, período de faturamento corrente.

Onde o tempo de banco foi gasto no mês

DELETE FROM net_http_response — coletor de lixo do pg_net	68,64%
realtime.list_changes	13,84%
Lista de conversas — 9 variantes da mesma query	8,94%
net.http_post — 621.495 chamadas	1,91%
DELETE net_http_response (2ª cópia, papel postgres)	1,59%
Todo o resto — inserts de mensagem, acks, crons, auth, IA...	5,08%

Somando tudo que é pg_net: 72,14% do tempo total de banco.

Percentuais de prop_total_time do pg_stat_statements, acumulados desde o último reset das estatísticas.

A consulta que come o banco

Chamadas	695.058
Tempo médio	439 ms
Pior execução	320 segundos
Tempo total	84,9 horas

Uma execução de 320 segundos é uma transação de 5 minutos segurando conexão, travando o autovacuum e empurrando o resto do banco para a fila.

Por que ela é lenta

net_http_response — tamanho físico vs. conteúdo útil

283 MB no disco
2.160 linhas vivas (~1%)

A tabela é 99% tupla morta. Todo DELETE varre 283 MB para achar quase nada.

O efeito em cascata — e por que isso explica o swap

A net._http_response é o **segundo maior objeto do seu banco**: 283 MB de 1,08 GB, ou **26% do total**. Ela compete pelos mesmos 906 MB de RAM que a messages . A sequência é:

6 crons por minuto
621 mil http_post

→

tabela incha
283 MB de bloat

→

DELETE fica lento
439 ms → 320 s

→

bloat expulsa a
messages do cache

→

swap
cache 0,983

Limpar essa tabela devolve 26% do banco. O banco cai de 1,08 GB para ~800 MB — e passa a caber nos 906 MB de RAM. É plausível que o swap de hoje desapareça sem trocar nada de máquina. Isso *não* torna o Micro suficiente para 98 números, mas torna a medição do teste de carga confiável.

93% das suas invocações de edge function são crons batendo em nada

27 jobs ativos no `cron.job`, dos quais 6 rodam a cada minuto.

Invocações por dia — de onde vêm

Total medido: 305.856 no período (~10.200/dia)

6 crons de 1 em 1 minuto	8.640
reprocess-inbound-events (2 min)	720
health-watchdog (5 min)	288
instagram-bind-next-post (5 min)	288
auto-close, purge-presence, horários	277

Total previsto pelos crons **10.213/dia**
× 28 dias = 285.964 de 305.856 medidas
≈ 93% da fatura é polling.

Metade desse polling é de um módulo bloqueado

Das 8.928 invocações/dia dos crons frequentes:

Instagram — 4.608	WhatsApp — 4.320
-------------------	------------------

51,6% do polling de minuto é do Wizzy Engage:
instagram-broadcast-send · instagram-flow-timeouts
instagram-process-followups · instagram-bind-next-post

O Engage está travado no App Review da Meta.
Esses 4 jobs rodam 4.608 vezes por dia para achar zero linha. Desligar é risco zero e ganho imediato.

Bônus: o job encerrar-enegnyork220826 (23/ago) já passou e vai disparar de novo em agosto de 2027. Remover.

A boa notícia que isso traz para o dimensionamento

O custo dos crons é **fixo**: 6 execuções por minuto hoje, 6 por minuto com 98 números. Ele não escala com a quantidade de números. Isso significa que os 72% de pg_net e os 93% de polling que você vê hoje **não vão multiplicar por 33** — o que multiplica é o trabalho de mensagem, que hoje é a menor fatia de tudo. Depois de limpar o pg_net e desligar o Engage, sobra um sistema cujo custo real é bem menor do que estes gráficos sugerem.

O que realmente muda de escala

DIMENSÃO	HOJE	ALVO (SET/2026)	FATOR	OBSERVAÇÃO
Números conectados	3	98	×33	Meta confirmada
Usuários simultâneos	5	100	×20	Pico de conexões Realtime
Mensagens por dia	~405	~13.200	×33	135 msg por número por dia
Pico de entrada	0,4 req/s	~13,6 req/s	×33	25 msg/min medidas → ~817 msg/min
Crescimento do banco	~25 MB/mês	~830 MB/mês	×33	2,06 KB por mensagem
Invocações de cron	10.213/dia	10.213/dia	×1	Custo fixo, não escala

Ajuste necessário no teste de carga

O alvo de **98 números e 100 usuários está correto** e é o que este documento dimensiona. O que precisa mudar é só um parâmetro do script: em `scripts/load-test-webhook.mjs:74`, `--rate` é **mensagens por segundo por instância**. O roteiro atual usa `--rate 2`:

Pico projetado para 98 números, a partir do seu medido

13,6 req/s

= 135 mensagens por número por dia (o seu real, ×33)

O que o roteiro testa com `--rate 2`

196 req/s = 172.800 mensagens por número por dia — 14× acima do pico real

Use dois testes: `--rate 0.3` (≈29 req/s, o dobro do pico projetado) como **critério de aprovação** do lançamento, contra as metas da Parte 4 de `docs/teste-de-carga.md`; e `--rate 2` por 5 minutos como **teste de ruptura**, só para descobrir o que quebra primeiro — sem valer como reprovação.

Compute Large — US\$ 110/mês, US\$ 100 líquidos a mais

	MICRO (HOJE)	MEDIUM	LARGE (RECOMENDADO)
RAM	1 GB	4 GB	8 GB
Tipo de CPU	compartilhada, burst	compartilhada, burst	dedicada
Conexões diretas	60	120	160
Conexões no pooler	200	600	800
I/O de base	11 MB/s	43 MB/s	79 MB/s
Teto de I/O	261 MB/s	261 MB/s	594 MB/s
Preço por mês	US\$ 10	US\$ 60	US\$ 110

Por que Large e não Medium

O motivo decisivo não é RAM — é o tipo de CPU. Micro, Small e Medium são *burstáveis*: entregam performance alta enquanto há crédito e despencam quando ele acaba.

Um CRM de WhatsApp tem carga **sustentada das 8h às 20h**, não picos curtos. É o perfil errado para burst. O Large é o primeiro nível com CPU dedicada — você troca uma queda invisível e imprevisível por um teto conhecido.

Em RAM, o Medium também é apertado no médio prazo: com 98 números o banco cresce ~830 MB por mês e passa de 4 GB em cerca de 4 meses — voltando ao swap de hoje, só que em produção cheia.

Até quando o Large serve

Com 98 números o banco chega a ~8 GB em cerca de um ano.

Projeção linear a 830 MB/mês, sem contar as purgas já existentes, que achatam a curva. Reavaliar o XL quando o banco passar de 5 GB.

Custo mensal no lançamento

ITEM	CONFIGURAÇÃO	US\$/MÊS
Supabase — plano	Pro	25
Supabase — compute	Large, menos US\$ 10 de crédito incluso	100
Supabase — disco	32 GB GP3 (24 GB acima do incluso, a US\$ 0,125/GB)	3
Lovable	Hospedagem do frontend estático	5
Evolution API	2 VPS de 8 vCPU / 16 GB (49 números cada)	70–140
Total		203–273
PITR — <i>não recomendado agora</i>	7 dias de retenção contínua	+100

Sobre o PITR: a revisão de escala manda ligar, e aqui a recomendação é o contrário, por custo-benefício. São US\$ 100/mês — dobrar a conta do Supabase para proteger 4 organizações. O plano Pro já inclui backup diário com 7 dias de retenção, e você tem a tabela `inbound_events` como caixa-preta da entrada. Ligue o PITR quando passar de ~20 organizações pagantes. A única razão para pagar antes disso é se perder até 24 horas de conversa já for inaceitável no lançamento.

Supabase e Evolution API

Supabase

Compute	Large — 8 GB, 2 núcleos dedicados
Disco	32 GB GP3 , autoscale ligado
PITR	desligado no lançamento
Pooler	Transaction mode para as edge functions; Session mode só para o dump do staging
Estatísticas	<code>pg_stat_statements_reset()</code> agora, reler em 48 h
Região	a mesma da Evolution — sem exceção

Evolution API — 2 máquinas de 49 números

A divisão não é por RAM, é por **raio de explosão**: uma Evolution única derruba os 98 números de todas as organizações ao mesmo tempo.

Por servidor (×2)	8 vCPU dedicada · 16 GB · 160 GB NVMe
Banco e cache	Postgres e Redis próprios , no mesmo host
Números	49 por máquina

As 8 vCPU não são para o regime normal — 49 números a 7 msg/s é pouco. São para a **tempestade de reconexão**, quando os 49 refazem o handshake criptográfico ao mesmo tempo. É o único pico real de CPU da Evolution.

Variáveis de ambiente obrigatórias na Evolution

```
CACHE_REDIS_ENABLED=true
CACHE_REDIS_SAVE_INSTANCES=true
DATABASE_SAVE_DATA_MESSAGE_UPDATE=false
DATABASE_SAVE_DATA_CONTACTS=false
DATABASE_SAVE_DATA_CHATS=false
WEBHOOK_EVENTS_PRESENCE_UPDATE=false
```

Redis não é opcional. Sem ele o store do Baileys vive no heap do Node e cada número custa ~300 MB. Com ele cai para ~120 MB: $49 \times 120 \text{ MB} \approx 6 \text{ GB}$, e os 16 GB ficam com folga real.

Os três `DATABASE_SAVE_DATA_*` desligam a cópia de mensagens, contatos e conversas dentro da Evolution — você já guarda tudo isso no Supabase, que é a fonte de verdade. Manter ligado é pagar disco e I/O por um espelho que ninguém lê.

Um item aberto: PRESENCE_UPDATE

O `zapi-configure-webhook/index.ts:103` assina esse evento hoje. Cada “digitando...” de cada contato vira uma invocação de edge function. Não deu para medir pelo `inbound_events` — ele só grava `messages.upsert` (98 registros em 24 h). **Mede assim:** Reports → Edge Functions, filtre por `zapi-webhook` e compare as invocações do dia com as ~400 mensagens/dia. Se estiver muito acima, desligue o evento. É o corte mais barato disponível.

Lovable: fica

Por US\$ 5/mês servindo estático que você mesmo builda e sobe, não existe conta que justifique migrar na véspera do lançamento. Ele **não processa carga nenhuma** — quem processa é o Supabase e a Evolution. A saída já está pronta se um dia quiser (`docker-compose.yml` + `docs/deploy-vps-docker.md`), sem perder o sync do repositório.

O risco real é o de sempre, e é gerenciável: ele commita direto na `main` junto com você e reverte alterações de RLS no sync. Dois guardrails, ambos já são a regra do projeto — **policy e RLS só entram pelo Lovable, e migration só à mão no SQL Editor**. Antes de cada deploy, `git log --oneline origin/main` para ver o que ele commitou em paralelo.

Sinal correlato no Query Performance: 4.760 execuções de `SELECT name FROM pg_timezone_names` com 0% de cache hit — é o PostgREST recarregando o schema cache, provavelmente disparado pelo sync. Custa 0,28% do banco. Não é urgente, mas explica um ruído constante.

O que fazer, na ordem, e por quê

A pergunta “troco o compute antes ou depois do teste de carga?” tem resposta: **antes** — mas não é o primeiro passo.

- 1 **Limpar o pg_net.** custo zero hoje
`TRUNCATE net._http_response;` seguido de `SELECT net.worker_restart();`, e um cron diário de `VACUUM (FULL) net._http_response`. Devolve 283 MB — 26% do banco — e ataca diretamente os 72% de tempo de banco desperdiçado.
- 2 **Desligar os 4 crons do Instagram** enquanto o Engage estiver travado no App Review.
`UPDATE cron.job SET active = false WHERE jobname LIKE 'instagram-%';` Corta 4.608 invocações por dia e metade do polling de minuto. Reative junto com o lançamento do Engage. Aproveite e remova o `encerrar-enegnyork220826`, que já passou.
- 3 **Aplicar a migration** `20260830180000`, que ainda está pendente à mão, e **resetar as estatísticas:** `SELECT pg_stat_statements_reset();`
- 4 **Esperar 48 horas e reler o Query Performance.** ponto de decisão
 Se o pg_net saiu do topo, você recuperou 70% do banco sem gastar nada — e a lista de conversas vira o novo número 1, que é o alvo certo para o próximo esforço de código. Confira também se o swap caiu.
- 5 **Subir para o compute Large.** +US\$ 100/mês
 Restart de cerca de 2 minutos, cobrado por hora e proporcional — errar o tamanho custa centavos. Vem antes do teste porque o staging tem que ter o *mesmo* compute da produção (`docs/criar-staging.md`): medir no Micro e depois subir invalida o resultado e obriga a refazer o staging inteiro.
- 6 **Montar o staging já no Large** pelo caminho do dump de schema, e rodar uma mensagem única antes de qualquer carga.
- 7 **Rodar o teste de aceitação** com `--rate 0.3 --duration 1800` (≈ 29 req/s, o dobro do pico projetado), contra as metas da Parte 4 de `docs/teste-de-carga.md`. este é o gate
- 8 **Rodar o teste de ruptura** com `--rate 2 --duration 300`, só para saber onde quebra e o que quebra primeiro. Não vale como reprovação.
- 9 **Derrubar o staging** assim que terminar — enquanto existir, ele custa outro Large.

O passo 1 é o de maior retorno do projeto neste momento

Não custa nada, não é código, não precisa de deploy e não depende do Lovable. E pode mudar o que o passo 7 vai medir: com 72% do banco ocupado varrendo lixo, qualquer teste de carga mede o lixo, não o seu sistema.

Consultas mais caras do período — as que sobram depois do pg_net

CONSULTA	CHAMADAS	MÉDIA	% DO BANCO	AÇÃO
Lista de conversas (9 variantes)	62.581	234–1.771 ms	8,94%	Alvo do próximo esforço de código
Uma variante sem filtro de organization_id	15.733	1.013 ms	3,58%	Achar quem ainda chama
wz_health_snapshot()	169	3.077 ms	0,12%	5,4% de cache hit — o vigia lê quase tudo do disco
contact_tags por contato	985.137	0,9 ms	0,20%	N+1 no frontend; barato por chamada
Ack de entrega — UPDATE messages	55.305	56–66 ms	0,77%	Conferir o índice em zapi_message_id
whatsapp_connection_logs INSERT	139.993	18 ms	0,58%	Provável resíduo pré-Semana 3
user_fingerprints INSERT	43.135	11 ms	0,11%	43 mil escritas para 7 usuários
pg_timezone_names (schema cache do PostgREST)	4.760	262 ms	0,28%	0% de cache; ruído do sync

Ressalva: o pg_stat_statements acumula desde o último reset, que nunca foi feito. Parte destes números é anterior às correções da Semana 3 — os 139.993 inserts em whatsapp_connection_logs, em particular. Por isso o passo 3 do plano é resetar e reavaliar em 48 h. Nada disso afeta o achado do pg_net, que é do worker da extensão e não do seu código.

Wizzy · Dimensionamento de infraestrutura para o lançamento de setembro/2026 · gerado em 31/08/2026.
Fontes: painel do Supabase (Reports, Usage, Advisors → Query Performance) em 31/08/2026; consultas diretas ao banco de produção; docs/REVISAO_ESCALA_LANCAMENTO.md ; docs/teste-de-carga.md ; docs/criar-staging.md ; tabela de compute e preços de supabase.com/pricing e da documentação de Compute & Disk. Item ainda em aberto: hospedagem atual da Evolution API (provedor, vCPU, RAM, Redis), que fecha a faixa de US\$ 70–140.